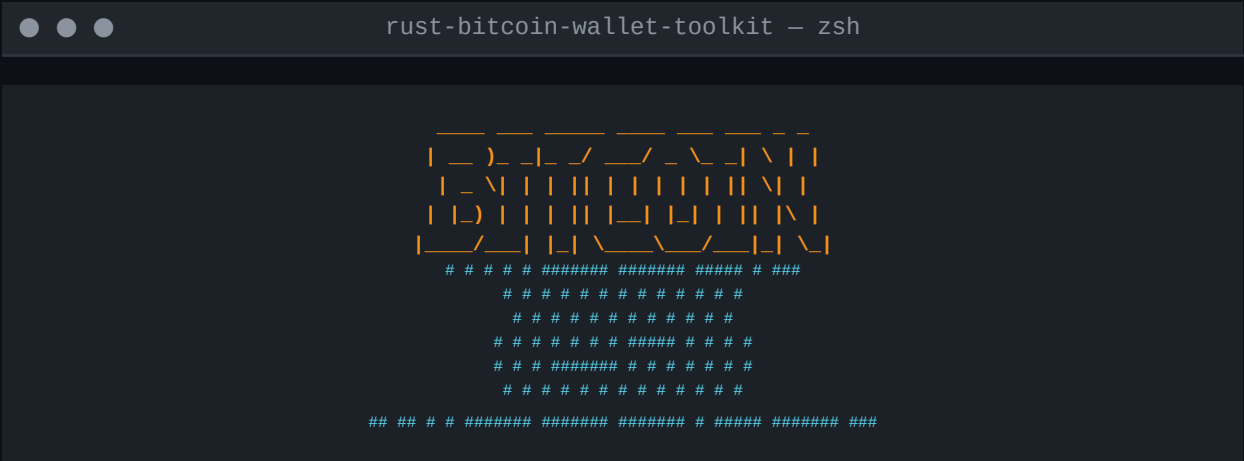




Rust Bitcoin Wallet CLI

Beginner's Toolkit — Moringa School AI Capstone

```
moringa@capstone:~/rust-bitcoin-wallet-toolkit$ cargo run --release
```

// technology	Rust + rust-bitcoin v0.31 + secp256k1
// focus	Bitcoin Development — Testnet Wallet CLI
// ai_tool	ai.moringaschool.com + Claude (claude.ai)
// prompts_used	9 prompts · 6 techniques from Moringa AI module
// tests	4 passing · correctness · randomness · format · determinism
// bonus	Themed ASCII banner CLI + Code Documentation Exercise
// repo	https://github.com/wandiamugo/rust-bitcoin-wallet-toolkit

Prompt-Powered Kickstart: Building a Beginner's Toolkit for Rust (Bitcoin Development)

01 // Title & Objective

```
const PROJECT = "Getting Started with Rust for Bitcoin Development";
```

A beginner-friendly toolkit documenting the complete process of learning Rust from scratch using AI-assisted prompting, culminating in a working Bitcoin testnet wallet CLI.

Technology: Rust **Focus:** Bitcoin Development **Background:** Python + JavaScript

~70% of critical CVEs in large codebases come from memory bugs Rust eliminates at compile time. For Bitcoin, bugs are not just crashes — they are financial losses.

02 // Quick Summary of Rust

what_is_it	A compiled, statically typed systems language with no garbage collector. Memory managed via compile-time ownership rules.
where_used	Bitcoin Core, Lightning Network (LDK), Firefox, Linux kernel, WebAssembly runtimes.
real_example	Bitcoin Dev Kit (BDK) powers wallets like Sparrow and Mutiny. Written entirely in Rust.
philosophy	"If it compiles, it's probably correct." The compiler is your co-author, not your adversary.

03 // System Requirements

os	macOS / Linux / Windows
editor	VS Code (recommended)
runtime	Rust v1.70+ via rustup — https://rustup.rs
tools	Cargo (included with Rust), Git

04 // Installation & Setup

```
// step 1 – install rust
```

```
$ ~  
  
curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh source $HOME/.cargo/env
```

```
// step 2 – make path permanent
```

```
$ ~  
  
echo 'source $HOME/.cargo/env' >> ~/.zshrc source ~/.zshrc
```

```
// step 3 – verify
```

```
$ ~  
  
rustc --version cargo --version
```

```
// step 4 – clone repo
```

```
$ ~
```

```
git clone https://github.com/wandiamugo/rust-bitcoin-wallet-toolkit cd  
rust-bitcoin-wallet-toolkit
```

// step 5 – install deps + build

```
$ ~
```

```
cargo add bitcoin@0.31 cargo add rand@0.8 cargo build # ~2-3 min first time
```

05 // Minimal Working Example

Generates a Bitcoin testnet wallet in 3 steps: (1) OS-RNG private key, (2) secp256k1 public key derivation, (3) P2PKH address + WIF encoding.

// run

```
$ ~  
  
cargo run
```

// src/main.rs – documented final version

```
$ ~  
  
// ===== // Bitcoin Key  
Generation Pipeline // OsRng ■■■ generate_private_key ■■■ derive_public_key ■■■  
derive_address // Each arrow is one-way – address reveals nothing about the public key, //  
public key reveals nothing about the private key. //  
===== /// Generates a  
cryptographically secure random private key. /// Panics probability ~1 in 2^128. Never log  
the SecretKey in production. fn generate_private_key(rng: &mut; OsRng) -> SecretKey { let  
mut key_bytes = [0u8; 32]; rng.fill_bytes(&mut; key_bytes);  
SecretKey::from_slice(&key_bytes).expect("not a valid secp256k1 scalar") } /// Derives  
compressed public key:  $P = k \times G$  (one-way, ECDLP). /// secp context passed in – never  
reconstruct per call. fn derive_public_key(secp: &Secp256k1, secret_key: &SecretKey;) ->  
PublicKey { let inner = PublicKey::from_secret_key(secp, secret_key); PublicKey {  
compressed: true, inner } // always compressed post-2012 } /// Derives P2PKH testnet  
address. Hardcoded to testnet – never send /// mainnet funds here. See docs for P2WPKH (~30%  
cheaper fees). fn derive_address(public_key: &PublicKey;) -> Address {  
Address::p2pkh(public_key, Network::Testnet) }
```

// expected output

```
$ ~  
  
Private Key (hex) : 331f449f8514957e8ea16942560d490b... Private Key (WIF) :  
cPJ5PGaiXSA3imcD1QKFuKWAaJWmzxKq... Public Key : 03b79e7d7e57b88d493349ba04b4a73c... Testnet  
Address : mudoXPzgDoXXhUoUWhYt9mhGRz5hEoYAeh // keys regenerated randomly every run – format  
always matches above
```

06 // AI Prompt Journal

9 prompts across 6 techniques from the Moringa AI curriculum modules: 'Learning a New Programming Language with AI' and 'Using AI to Help with Testing' and 'Code Documentation Exercise'.

prompt #1 — Conceptual Understanding

// Section 1.1 — Language Learning

prompt_used:	What are the key philosophical differences between Rust and Python/JS? What problems was Rust designed to solve?
ai_summary:	Rust philosophy: 'If it compiles, it's probably correct.' Ownership over shared refs, Result over exceptions, immutability by default. For Bitcoin: type safety prevents key/address confusion.
helpfulness:	<i>Very helpful — comparison table + Bitcoin-specific context made concepts immediately relevant.</i>

prompt #1b — Follow-up: Compile vs Runtime

// Section 2.2 — Deep Understanding

prompt_used:	What is the difference between runtime and compile time? In simple terms?
ai_summary:	Recipe analogy: compile time = editor catching errors before print; runtime = chef finding missing ingredients mid-service.
helpfulness:	<i>Very helpful — recipe analogy made an abstract CS concept immediately clear.</i>

prompt #2 — Step-by-Step Breakdown

// Section 1.2 — Language Learning

prompt_used:	Break down ownership, borrowing (& vs &mut;), Option/Result, and Rust structs vs Python classes.
ai_summary:	Ownership = one owner, freed on scope exit. & = multiple readers. &mut; = one exclusive writer. Option replaces null. Result replaces exceptions.
helpfulness:	<i>Extremely helpful — car keys + book lending analogies made abstract concepts concrete.</i>

prompt #3 — Guided Implementation

// Section 1.3 — Language Learning

prompt_used:	Guide me through implementing a Rust Bitcoin testnet wallet. Explain syntax vs Python.
ai_summary:	Fully annotated three-function wallet. Key lessons: let vs let mut, & borrows, return without return keyword, :: vs . operator.
helpfulness:	<i>Extremely helpful — every line had a Python/JS equivalent comment.</i>

prompt #4 — Understanding Verification

// Section 1.4 — Language Learning

prompt_used:	Review my code. Verify best practices, find improvements, spot Python/JS habits.
ai_summary:	Verdict: Solid First Rust. Issues: Secp256k1 recreated per call, .expect() overuse, rng scoped inside function.
helpfulness:	<i>Most actionable — identified real performance issues traced to Python/JS habits.</i>

prompt #5 — Learning Through Teaching

// Section 2.3 — Language Learning

prompt_used:	I explain Rust to another developer — verify what I got right, wrong, and missing.
--------------	--

ai_summary:	Ownership/borrowing/Option/Result all correct. Correction: traits share behaviour not data. Missing: ? operator, lifetimes.
helpfulness:	<i>Most valuable — Feynman technique revealed exactly where understanding was solid vs approximate.</i>

prompt #6 — Test Improvement Guidance // Testing Module — Section 2

prompt_used:	I wrote a basic test. Guide me to improve it without writing tests for me.
ai_summary:	Guided questions revealed 3 more behaviours: randomness, format (m/n prefix), determinism. Result: 4 passing tests.
helpfulness:	<i>Guided approach worked — questions revealed what was actually worth testing.</i>

prompt #7 — Comprehensive Documentation // Code Documentation — Prompt 1

prompt_used:	Create comprehensive /// doc comments for all three wallet functions including params, returns, panics, security, and examples.
ai_summary:	Formal /// sections with curve order in hex, probability ~ 1 in 2^{128} , security rules (OsRng only, zeroize, never log), address format table, runnable assert! examples.
helpfulness:	<i>Excellent API surface documentation. Weak on inline logic — that required Prompt 2.</i>

prompt #8+9 — Intent, Logic + Combined // Code Documentation — Prompts 2 & 3

prompt_used:	Document the intent and logic behind the code, then combine both versions into one final production-quality result.
ai_summary:	Prompt 2 added: top-level pipeline comment, OS syscall names (getrandom/BCryptGenRandom), $P=k \times G$ explanation, compressed key assumption risk, CompressedPublicKey newtype improvement suggestion. Prompt 3 merged both into final balanced version.
helpfulness:	<i>The two-prompt approach revealed that contract (Prompt 1) and reasoning (Prompt 2) are genuinely different things — neither alone produces complete documentation.</i>

07 // Code Documentation Exercise

Applied the two-prompt documentation technique from the Moringa Code Documentation Exercise to produce production-quality Rust documentation.

// comparison

aspect	prompt_1	prompt_2
style	Formal /// API reference	Inline // logic comments
best_strength	Params, returns, security rules, address table	Pipeline diagram, OS syscalls, improvement suggestions
missing	No inline body comments	Less formal on params/returns
verdict	Documents the CONTRACT	Documents the REASONING

// key insight

The CompressedPublicKey newtype suggestion was only discovered through careful documentation — the implicit contract between `derive_public_key()` and `derive_address()` had no compiler enforcement. Writing it down made the design gap immediately visible. This is why documentation is not just writing — it is a design tool.

// final rule

Comment the WHY, not the WHAT. If the code is readable, silence is better than a comment that restates it. Three tiers: (1) top-level architecture comment for the system shape, (2) /// doc comments for non-obvious contracts and security warnings, (3) inline // only for genuinely surprising lines.

08 // Common Issues & Fixes

// error	// cause	// fix
command not found: cargo	PATH not loaded after install	<code>source \$HOME/.cargo/env</code>
cargo build: 0.01s, no output	Dependencies missing from Cargo.toml	<code>cargo add bitcoin@0.31 &&</code> <code>cargo add rand@0.8</code>
cargo test: running 0 tests	File not saved correctly by VS Code	<code>cat > src/main.rs</code> (write directly from terminal)
Secp256k1::new() inside fn	Recreates expensive lookup tables each call	Create once in main(), pass as &secp;
.expect() crashes on error	No recovery — full panic on failure	Replace with ? operator + Result propagation

09 // Tests

```
$ ~
```

```
cargo test
```

```
$ ~
```

```
running 4 tests test test_two_keys_are_different ... ok test
tests::test_private_key_generation ... ok test test_testnet_address_format ... ok test
test_public_key_derived_from_known_private_key ... ok test result: ok. 4 passed; 0 failed; 0
ignored; 0 measured
```

test_name	what_it_verifies
test_private_key_generation	Private key is exactly 32 bytes
test_two_keys_are_different	Random generation never produces duplicate keys
test_testnet_address_format	Testnet P2PKH address always starts with 'm' or 'n'
test_public_key_from_known_key	Same private key always yields same public key (determinism)

10 // Learning Reflections

// ownership	The car keys analogy: when you hand keys to someone, you no longer have them. That is Rust's move semantics. Python passes references freely. Rust makes every transfer explicit.
// borrow_checker	Early compiler errors felt like obstacles. By Prompt #4 they were surfacing real design problems. Every time it rejected code, it was right. Ask 'what is it seeing?' not 'how do I get past this?'
// immutability	Writing let instead of let mut by default made me conscious of which values actually need to change. Python mutates everything freely. Rust inverts this instinct entirely.
// explicit_errors	Python exceptions are invisible in function signatures. Rust's Result means every fallible function declares it upfront. For Bitcoin wallet code this is not optional.
// traits_not_inherit	Traits share behaviour only — not data. For data sharing, Rust uses composition: contain a struct, do not inherit from it.
// documentation	Comment the WHY not the WHAT. The two-prompt documentation technique revealed that contract and reasoning are different things — Prompt 1 documents what a function promises, Prompt 2 documents why each line exists.
// ai_as_tutor	Nine structured prompts across six techniques took me from zero Rust to a working, documented, tested Bitcoin wallet CLI. The Feynman-technique prompt was most valuable.

11 // References

// official_docs

- > The Rust Book <https://doc.rust-lang.org/book/>
- > Rust Std Library <https://doc.rust-lang.org/std/>
- > Cargo Book <https://doc.rust-lang.org/cargo/>

// bitcoin_crates

- > rust-bitcoin v0.31 <https://docs.rs/bitcoin/0.31.2/bitcoin/>
- > secp256k1 <https://docs.rs/secp256k1/latest/secp256k1/>
- > rand 0.8 <https://docs.rs/rand/0.8.5/rand/>

// learning

- > Rustlings <https://github.com/rust-lang/rustlings>
- > Rust By Example <https://doc.rust-lang.org/rust-by-example/>
- > Rust for Rustaceans <https://nostarch.com/rust-rustaceans>

// bitcoin_dev

- > Bitcoin Dev Kit <https://bitcoindevkit.org>
- > Learn Me a Bitcoin <https://learnmeabitcoin.com>
- > Bitcoin Dev Guide <https://developer.bitcoin.org/devguide/>

// community

-> Rust Users Forum <https://users.rust-lang.org>

-> Bitcoin Stack Exchange <https://bitcoin.stackexchange.com>

-> Rust Discord <https://discord.gg/rust-lang>

// project_repo

-> GitHub Repository <https://github.com/wandiamugo/rust-bitcoin-wallet-toolkit>

// end of document

built with Rust + Bitcoin + AI @ Moringa School